HOMEWORK 1 • DLE-AI-202

Deep Learning

Neural-Network Foundations in PyTorch (Weeks 1--2)

AI Academy · National Artificial Intelligence Center

Released	Sun 1 Nov 2026	**Points**	100 (2.5% of grade (HW1 to HW4 = 10%))
Due	**Sun 15 Nov 2026, 23:59 Baku**	**Submit via**	`.zip` on Moodle (§8)

1 Overview and learning objective

Build, check and train a neural network with PyTorch, and understand every line. This homework tests course objective **DL1**: turn the mathematics of Weeks 1–2 into a working model that *learns*, and diagnose it when it does not. Week 1 gives the model, its maximum-likelihood loss, activations and initialisation; Week 2 gives backpropagation, optimisers and regularisation. You will use PyTorch's tensors and `nn.Module`, but you will also write the pieces that frameworks usually hide (stable losses, a manual backward pass, a training loop), so that nothing is a black box.

- **Part A: Written (30 pts).** Eight short problems: the equations your code implements, one full backward pass by hand, and debugging reasoning.
- **Part B: Code (45 pts).** Complete the stubs in `src/` until the provided `pytest` checks pass: tensor basics and losses (B1), manual backpropagation and a custom autograd `Function` (B2), an `MLP` module with initialisation (B3), training utilities (B4), diagnostics with hooks (B5).
- **Part E: Experiments (15 pts).** Four short experiments in the notebook: depth and initialisation, optimisers, regularisation on noisy data, and a final model with test accuracy ≥ 0.95.
- **Part C: Reflection (10 pts).** Three prompts about *your own* code and results, and your AI-tool disclosure.
- **Bonus (+10).** `MyAdam` as a `torch.optim.Optimizer` (+6) and MixUp (+4). The total is capped at 100.

Estimated effort: 10–12 hours over two weeks. After Week 1: A1–A5, B1, B3, B5 and experiment E1. After Week 2: A6–A8, B2, B4, E2–E4.

What you start from (the homework kit)

`hw1_notebook.ipynb` (the notebook you work in), `torch_cheatsheet.ipynb` (a runnable PyTorch reference: tensors, autograd, `nn.Module`, losses, optimisers, schedulers, data loaders, a complete training loop, diagnostics, common errors), `src/` (stubs and provided helpers), `tests/` (the checks), `report_template.tex`, `pledge.txt`, `requirements.txt`, `README.md`. The dataset (scikit-learn digits, 8×8 images, 10 classes) is built in: there is nothing to download and everything runs on a CPU in minutes.

What you submit (one zip)

(1) `report.pdf`: the filled-in report template (all written answers). **(2)** your code: `src/` and the notebook `hw1_notebook.ipynb` *with outputs*. **(3)** `figures/final_curves.png` (made by the notebook) and a signed `pledge.txt`. Details in §8.

2 How this homework is marked

Per the course syllabus, homework is **marked for a complete, good-faith attempt**, not for correct answers, and every set comes back with written feedback. *Full credit:* every problem attempted, with the working and the code shown. *Half credit:* a substantial attempt with problems missing. *No credit:* missing, token or copied work. The point values below show where the effort goes and what the feedback will comment on. The `pytest` checks are **your own feedback**: they tell you whether your code is right, so use them. *Homework is where you are allowed to be wrong; the quizzes and the exams are where correctness counts*, and they ask the same kind of questions.

3 Part A: Written (30 pts)

Show every step. Typeset the maths, or embed a clean scan of handwritten work in the report template. A bare final number earns no feedback; the reasoning is what we read.

Problem 1 Shapes and parameter count [3 pts]

An MLP has layers $20 \to 64 \to 32 \to 5$ with ReLU hidden units. A batch of 128 examples goes through it. (a) Give the shape of $W^{(\ell)}$ and $\mathbf{b}^{(\ell)}$ as stored by `nn.Linear`, and of $\mathbf{z}^{(\ell)}$ and $\mathbf{a}^{(\ell)}$ for the batch, for every layer. (b) Count the parameters. (c) Count the multiply–add operations of one forward pass for *one* example.

Problem 2 Losses come from likelihood [4 pts]

(a) Assume $y = \hat{y} + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2)$. Derive the negative log-likelihood of one example and show that minimising it over the data is minimising the mean squared error. (b) For a logit z, $\hat{y} = \sigma(z)$ and label $y \in \{0, 1\}$, show that $-[y \log \hat{y} + (1-y)\log(1-\hat{y})] = \max(z, 0) - zy + \log\!\left(1 + e^{-|z|}\right)$ (the stable form you code in B1), and say why this form never overflows. (c) Show that $\partial L / \partial z = \hat{y} - y$.

Problem 3 Backpropagation by hand [6 pts]

A $2 \to 2 \to 2$ network: ReLU hidden layer, linear output, softmax and cross-entropy. With

$$W^{(1)} = \begin{pmatrix} 1 & -1 \\ 2 & 0 \end{pmatrix},\ \mathbf{b}^{(1)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix},\ W^{(2)} = \begin{pmatrix} 1 & 1 \\ -1 & 2 \end{pmatrix},\ \mathbf{b}^{(2)} = \mathbf{0},\ \mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix},\ \mathbf{y} = (1, 0),$$

and $\mathbf{z}^{(1)} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$, $\mathbf{h} = \mathrm{ReLU}(\mathbf{z}^{(1)})$, $\mathbf{z}^{(2)} = W^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$:

(a) Compute $\mathbf{z}^{(1)}, \mathbf{h}, \mathbf{z}^{(2)}, \mathbf{p} = \mathrm{softmax}(\mathbf{z}^{(2)})$ and L. [1]

(b) Draw the computational graph, then derive and evaluate $\boldsymbol{\delta}^{(2)} = \partial L/\partial \mathbf{z}^{(2)}$, $\partial L/\partial W^{(2)}$ and $\partial L/\partial \mathbf{b}^{(2)}$. Name the chain-rule factors *before* multiplying. [2]

(c) Continue one layer down: $\boldsymbol{\delta}^{(1)}$ (mind the ReLU gate), $\partial L/\partial W^{(1)}$, $\partial L/\partial \mathbf{b}^{(1)}$. [2]

(d) Which hidden unit receives *no* gradient for this example, and why? What would happen if that were true for *every* example (the dying-ReLU problem)? [1]

Check your numbers with the notebook cell in B2 only *after* you have written them.

Problem 4 Activations and vanishing gradients [4 pts]

(a) Show $\sigma'(z) = \sigma(z)(1-\sigma(z))$ and find its maximum. (b) Ignoring the weights, the gradient reaching layer 1 of an L-layer sigmoid network is at most 0.25^{L} times the gradient at the output. How many layers make this smaller than 10^{-6}? (c) Derive the derivative of Swish, $f(z) = z\sigma(z)$ (you code it in B2), and of GELU, $f(z) = z\Phi(z)$, in terms of Φ and the standard normal density φ.

Problem 5 Initialisation [4 pts]

(a) A unit computes $z = \sum_{j=1}^{n} w_j a_j$ with independent, zero-mean w_j and a_j. Show $\mathrm{Var}(z) = n\,\mathrm{Var}(w)\,\mathbb{E}[a^2]$, and use $\mathbb{E}[a^2] = \frac{1}{2}\mathrm{Var}(z_{\text{prev}})$ for ReLU to derive $\mathrm{Var}(w) = 2/n$ (He). (b) Give the He and the Xavier standard deviation for a $784 \to 256$ layer. (c) Why can a network not be initialised with all weights equal to zero? (d) What loss do you expect from a freshly initialised 10-class network and why? Which two bugs make it much larger?

Problem 6 Optimisers [4 pts]

Minimise $f(w) = 5w^2$ from $w_0 = 1$ with learning rate $\eta = 0.05$. (a) Do two steps of plain SGD and two steps of momentum ($v \leftarrow \gamma v + \eta g$, $w \leftarrow w - v$, $\gamma = 0.9$, $v_0 = 0$). What do you notice about the momentum iterates? (b) Write Adam's first update for a gradient $g_1 = 0.2$ with learning rate η, including the bias correction, and say how large the step is. (c) Why is weight decay applied *outside* the adaptive scaling in AdamW?

Problem 7 Regularisation [3 pts]

(a) Show that SGD on $L(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|^2$ equals shrinking the weights by $(1-\eta\lambda)$ and then taking the usual gradient step. (b) With inverted dropout (drop with probability p, scale kept units by $1/(1-p)$), show $\mathbb{E}[a'] = a$. (c) Why must batch normalisation use different statistics at training and inference time?

Problem 8 Debugging [2 pts]

For each symptom give the most likely cause and the *first check* you would run. (i) 10-class loss stays at 2.30 and never moves. (ii) Loss becomes `NaN` after 50 steps. (iii) Training accuracy

99%, validation accuracy 70%. (iv) Validation accuracy differs every time you evaluate the same model on the same data.

4 Part B: Code (45 pts)

Work in `src/`. Each module has a docstring that specifies exactly what to implement; the tests in `tests/` compare with PyTorch's own functions or with finite differences. Run them from the project root with `pytest`, or from the notebook cells provided. *Run the tests early and often.*

Part	File	What you implement	Pts
B1	`tensor_basics.py`	`standardize`, `one_hot`, `pairwise_sq_dists` (no loops), stable `softmax`, `log_softmax`, `cross_entropy`, `bce_with_logits`, `mse` from raw tensor operations	10
B2	`manual_mlp.py`	forward, *manual* backward of a 2-layer MLP (the backpropagation equations of Week 2), finite-difference `numerical_grad`, `SwishFn` as an `autograd.Function`	10
B3	`model.py`	`MLP(nn.Module)` with activation, dropout, batch/layer norm; `init_weights` (He, Xavier, normal, zeros) from the variance formulas; `count_params`	8
B4	`train.py`	`warmup_cosine`, `train_one_epoch`, `evaluate`, `fit` with early stopping and best-weights restore	12
B5	`diagnostics.py`	forward-hook `layer_stats`, `dead_relu_fraction`, `overfit_one_batch`	5

Rules for B1 and B2. Inside your own functions use tensor operations (indexing, broadcasting, matmul, `exp`, `log`, `logsumexp`). Do not call `F.cross_entropy`, `torch.softmax`, `F.one_hot` and similar *inside the answers*: the tests use them as the reference. In B2 do not call `.backward()` in `backward`; that is the point. From B3 on, use PyTorch freely, but keep `self.net` (an `nn.Sequential`) and the function signatures: the tests and the diagnostics rely on them.

Speed tip. Tiny networks run fastest on *one* CPU thread; with many threads PyTorch can be a hundred times slower. The kit sets one thread for you (`DL_HW_THREADS=N` to change it). The cheatsheet shows a measurement.

5 Part E: Experiments (15 pts)

In the notebook, using your own code from Part B and the seed derived from your Academy ID (`STUDENT_ID`). Each experiment ends with a short written answer (3–5 sentences, your own words, citing *your* numbers); copy these into the report.

1. **E1 (4 pts) Depth and initialisation.** A 10-layer MLP built four ways (sigmoid+Xavier, ReLU+tiny normal, ReLU+Xavier, ReLU+He). Measure per-layer output std and gradient norm at initialisation with `layer_stats`, then train with SGD. Explain which configurations fail and why.

2. **E2 (3 pts) Optimisers.** SGD, SGD+momentum, RMSprop, Adam and AdamW on the same model, data and seed. Compare validation-loss curves; say what differs and why one learning rate cannot rank optimisers.

3. **E3 (4 pts) Regularisation.** 300 training images with 25% corrupted labels so that overfitting is visible. Compare baseline, dropout, batch norm, weight decay, early stopping and

a combination in a table and a plot; say what each mechanism does.

4. **E4 (4 pts) Final model.** Choose a configuration *using the validation set only*, train with warm-up + cosine and gradient clipping, and evaluate on the test set *once*. Target: test accuracy ≥ 0.95. Save `figures/final_curves.png`.

6 Part C: Reflection (10 pts)

Answer in the report (3–5 sentences each), referring to *your own* code and numbers. These rehearse the short oral verification of homework that begins in Week 7: *work you cannot explain is not your work.*

1. **[4 pts]** Trace one weight's gradient through your `manual_mlp.backward`, line by line, and say where the activation derivative enters.
2. **[3 pts]** Which two lines of your training or evaluation code would silently break things if removed (e.g. `zero_grad`, `model.eval()`, `no_grad`)? What exactly goes wrong?
3. **[3 pts]** One result in Part E that surprised you or differed from the lecture, and your explanation. Then your **AI-tool disclosure**.

7 Bonus: optional (up to +10)

Total capped at 100. In `src/bonus.py`: **[+6]** `MyAdam`, Adam with bias correction as a `torch.optim.Optimizer` subclass matching `torch.optim.Adam` to 10^{-6} (`tests/test_bonus.py`); **[+4]** `mixup_batch`, then train with it in the E3 setting and add a row to the table. A bonus that breaks the core tests will not be counted.

8 Submission

One archive on Moodle named `HW1_<STUDENT_ID>.zip` containing:

1. `report.pdf`, the compiled `report_template.tex`, with all written answers (Parts A, E, C);
2. `src/` (your completed code) and `hw1_notebook.ipynb` **with all outputs saved** (run *Restart & Run All* before saving);
3. `figures/final_curves.png` and the signed `pledge.txt`.

We run `pytest` on your `src/` and read your notebook. Do not submit data files or the cheat-sheet.

Deadline, late work, appeals

Due **Sunday 15 November 2026, 23:59 Baku time** (two weeks after release), on Moodle. Late submissions and extensions follow the course policy: see *Course Policies*, *Homework, labs, and late submissions* in the syllabus (extensions must be requested *before* the deadline). Grading queries and appeals follow *Grades, feedback, and appeals*.

Integrity and AI tools

Your submission must be your own work and reflect your own understanding. Per *Course Policies*, *Academic integrity and the use of AI tools*: AI assistants are welcome for learning and debugging, but you must be able to explain and reproduce everything you submit without them, and you note where one was used (Part C3 and `pledge.txt`). From Week 7 a random sample of homework is verified in a short oral conversation.

Tip. Run the B1 test first, then B3 and B5, and do E1 as soon as they pass: it makes the Week-1 theory visible. Do the *by hand* problems A3 before using the check cell. When a result looks wrong, follow the debugging order of the cheatsheet: shapes and dtypes, initial loss $\approx \ln K$, overfit one batch, learning rate, gradient statistics, `train()`/`eval()`, and only then change the model.